

WRO 2026 Engineering Journal – Team LEGACY

Category: Future Engineers

Country: Ghana

1. INTRODUCTION & TEAM OVERVIEW

Team Name: Legacy

Participants: Alfredina Abigail Dzagli, Ekow Gyandoh Asiedu, Godsway Kwame Ahlijah

Coach: Manasseh Sassure-Saddick

Institution: MIKROBOT ACADEMY

2. MOBILITY AND MECHANICAL DESIGN

2.1 Chassis & Steering Justification

A. We initiated the design by leveraging an existing RC car chassis. It came with two prebuilt DC motors (one for drive and one for steering) and an integrated gear system. This decision to choose an existing RC Chassis to a Custom 3D Printed Chassis was driven by the following reasons:

- **The need for a durable, pre-aligned mechanical foundation:** Unlike a 3D-printed chassis, which is built in layers that can easily split and fracture during a crash or fall, a factory-moulded chassis is mostly one solid piece of tough plastic, making it much stronger and highly resistant to breaking on impact.

Custom 3D prints are highly prone to warping and tolerance errors, which leads to structural misalignment and material waste from repeatedly re-printing poorly fitted components like gears and gear ratios. In contrast, our factory-moulded chassis features a pre-aligned foundation with a built-in, integrated gear system. This eliminates mechanical backlash and steering drift, ensuring precise, repeatable tracking for autonomous navigation.

- **4-Wheel Drive (Integrated Gear System):** To comply with the **WRO Future Engineers General Rules**, which mandate a vehicle kinematics scheme different from a standard differential drive system, we utilized a chassis with a car-like steering configuration with an integrated 4WD mechanical foundation.

Technical Parameter	Integrated 1-Motor 4WD (Our System)	Multiple-Motor Differential Drive
Motor Configuration	Optimized. Uses a single efficient drive	Complex. multi-motor setups require

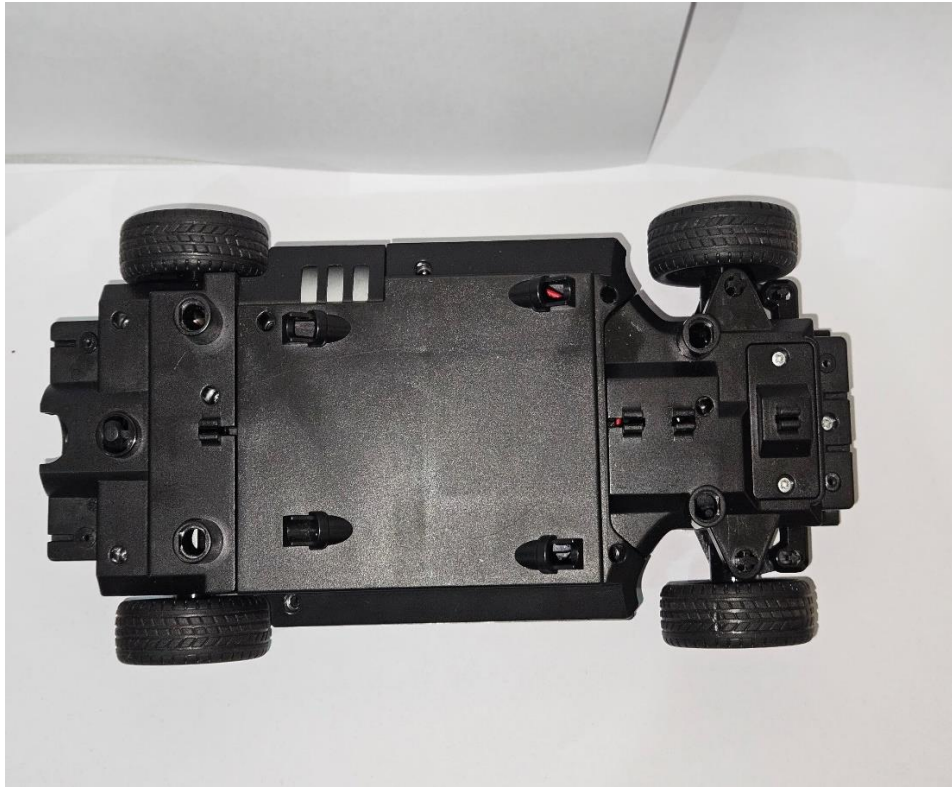
	motor and one servo, drastically reducing hardware complexity.	extra motor drivers, increasing bulk and failure points.
Power Budget & Consumption	Low. Draws current for only one electrical motor load, optimizing battery life.	High. Multiple motors running simultaneously draws significant combines current.
Speed Synchronization	Flawless. Synchronized naturally by the rigid gear train, eliminating software lag or tuning errors.	Flawed Inherently relies on heavy code processing (PID loops) to correct constant motor speed mismatches (Siegwart et al., 2011).
Operational Traction	High Distributes motor torque across all four contact surfaces, maximizing mat grip (Gillespie,1992).	Variable High tendency for wheel slippage and skid errors during tight turns (Siegwart et al.,2011).

- **Chassis Weight Baseline:**

The structural chassis—including the two integrated motors—weighs exactly 0.20 kg. This serves as a critical baseline for tracking total vehicle weight as the remaining electronic components are added, ensuring the completed robot remains safely within competition threshold limits and maintains optimal stability.

B. Our autonomous vehicle prototype utilizes an Ackermann steering geometry integrated directly into the physical chassis design. The following are the reasons why the Ackermann steering is preferred:

- **Path Predictability:** Unlike differential or tank steering configurations, the Ackermann mechanism provides a smooth, stable, and highly predictable trajectory. This is critical for maintaining an undisturbed computer vision and sensor tracking feed.
- **Reduced Friction & Wear:** By allowing the inner and outer wheels to naturally follow different turning radii during a corner, the design eliminates tire "scrubbing." This reduction in lateral friction directly prevents premature tire wear while minimizing structural stress on the steering linkages and chassis.



2.2 Torque/Speed Reasoning

In designing our autonomous vehicle, torque is the turning force the motors must generate to break static friction and get the car moving, while speed (RPM) determines how fast it can navigate the course. Ideally, we want high amounts of both, but real-world hardware constraints dictate how much power actually reaches the wheels. To understand how these two factors interact on our vehicle, we must analyze the specific electromechanical bottlenecks and design choices affecting our powertrain:

- **The Impact of the L298N Voltage Drop**

During initial testing, the bot was running well, exhibiting excellent torque and speed when the battery was connected straight to the motor, until the introduction of control circuitry added internal resistance and voltage drops across the driver components. Specifically, when integrating the control hardware, our 7.4V battery setup struggled significantly to move the vehicle effectively. This issue stems from the fact that the L298N motor driver relies on an older BJT-based H-bridge design. According to the manufacturer's technical specifications, this architecture introduces a severe total internal voltage drop (V_{CEsat}) combining the high-side and low-side saturation losses, which typically amounts to 1.8V at a 1A load and can peak as high as 3.2V (STMicroelectronics 2000, p. 3).

Consequently, the actual voltage reaching the motor terminals was choked down to a range between approximately 5.6V and 4.2V. Because torque in a DC motor is directly proportional to the operating voltage, this massive voltage drop severely diminished the available electrical torque. As a result, the motors routinely stalled; the crippled voltage simply could

not supply the high starting current, which is typically 4 to 5 times higher than the running current necessary to overcome the motor's static friction and initiate motion.

To illustrate how the operating voltage dictates this behaviour, we can look at the simplified relationship between a DC motor's torque and its input voltage (Bird 2017, p. 385):

$$T = k.V$$

Where:

T = Electromagnetic torque

k = A constant combining the motor's internal resistance, structural design, and magnetic field flux

V = Applied operating voltage

- **Gear Ratio Justification**

Because the vehicle utilizes an existing RC chassis, it relies on a pre-installed integrated gear system to manage power delivery. This gear ratio acts as a critical mechanical trade-off, systematically reducing the high raw speed (RPM) of the DC motors down to a practical, controllable operating speed while simultaneously multiplying the output torque delivered directly to the drive wheels. Furthermore, manual inspection verified that the drivetrain is completely free of mechanical binding. Ensuring high mechanical efficiency is vital, as any internal binding would artificially spike the torque required to rotate the wheels, forcing the motors to draw excessive stall currents capable of overheating and melting the control circuitry wiring.

- **Power System Upgrade Justification**

To overcome the severe losses of the L298N driver, the power system was upgraded to a series configuration of three 3.7V (1800mAh) 18650 Li-ion cells housed in a 3-slot series battery holder, providing a total nominal voltage of 11.1V (3S). This transition provides critical voltage headroom; accounting for the driver's internal BJT saturation losses, a typical 1.8V drop leaves a substantial 9.3V at the motor terminals, and even under peak 3.2V drop conditions, the motors still receive a viable 7.9V. Because motor torque and RPM scale linearly with operating voltage, this elevated potential difference guarantees the high-performance throughput required for the vehicle. The resulting "aggressive torque" and elevated speed are absolutely vital for overcoming static friction instantly, executing sharp, precise steering adjustments, and meeting the rigorous demands of the timed WRO competitive runs.

- **Control System Power Isolation Justification**

To ensure system reliability, a dual-rail power routing strategy was implemented to completely isolate the logic circuitry from the drivetrain electromechanical loads. Microcontrollers and single-board computers like the Raspberry Pi 3 are highly sensitive to

voltage fluctuations. If the Raspberry Pi shared a power rail with the L298N motor driver, the heavy current spikes drawn by the DC motors during rapid acceleration or stalls would cause severe voltage sags (brownouts), leading to spontaneous system resets or file corruption. To eliminate this risk, the Raspberry Pi is powered independently via a dedicated 2-slot 18650 battery supply, guaranteeing a stable, noise-free voltage rail for processing vision and steering algorithms.

Parameter	7.4V Battery System	11.1V 18650 Li-ion Series System (Upgrade)
Input Voltage	7.4V	11.1V (3×3.7V Cells in Series)
Driver Voltage Drop	1.8V (typical) to 3.2V (peak)	1.8V (typical) to 3.2V (peak)
Voltage at Motor	~5.6V down to 4.2V	~9.3V down to 7.9V
Effect on Torque	Insufficient to overcome static friction (stalling)	High torque for rapid acceleration
WRO Advantage	Poor reliability; slow response	Consistent performance and timed-run efficiency

2.3 Design Iteration:

We followed an iterative "plan-build-test-improve" cycle to refine our vehicle's mechanical stability and power delivery. Our chassis underwent three major versions:

Version 1: The Stock RC Prototype

- **Hardware:** We repurposed a standard toy RC car chassis with two prebuilt DC motors and a 4-wheel drive system. The initial setup used an Arduino Uno and L298N driver powered by the stock **7.4V 600mAh battery**.
- **Observation:** During floor testing, the vehicle exhibited "jerking" motions and failed to move.
- **Engineering Analysis:** Diagnostic testing confirmed that the L298N's internal voltage drop (~2V) reduced the battery output to ~5.4V at the motor terminals. This was insufficient to overcome the high **stall current** (4 to 5 times running current) required to break static friction.
- **Result:** This version was mechanically non-functional for the competition due to chronic stalling

Version 2: Power System Integration

- **Modification:** To overcome the voltage bottleneck, we upgraded to a series configuration of three 3.7V 18650 Li-ion cells housed in a 3-slot battery holder to deliver a nominal 11.1V, yielding an effective range between approximately 9.3V and 7.9V to the motors depending on driver load.
- **Mechanical Challenge:** The 3-slot 18650 battery holder occupied a significantly larger physical space and volume, preventing it from fitting inside the chassis' original factory battery compartment.
- **Iteration:** Instead of altering the internal structure, we kept the chassis intact and mounted the large battery holder externally on the main deck, fastening it securely directly on top of the original factory battery slot.
- **Trade-off Analysis:** While this higher mounting point raised the centre of mass—potentially impacting lateral stability—the gain in "aggressive torque" was a necessary trade-off for WRO timed-run efficiency and reliable acceleration.

Version 3: Final Vision & Mechanical Refinement

- **Status:** Integrated a Raspberry Pi 3, an Arduino, and a Raspberry Pi Camera Module 3 NoIR Wide for autonomous lane detection.
- **Structural Modification:** To overcome severe spatial constraints, we designed and 3D-printed two custom structural components:
 1. **Dual-Tier Flat Deck (Red):** Mounted directly on top of the original chassis to act as an upper component deck. This safely houses the Raspberry Pi, Arduino, and L298N driver, and an additional 2-slot battery holder dedicated to powering the Raspberry Pi independently on top, while creating a protective lower clearance area where the new 18650 battery holder sits neatly underneath on the main chassis floor.
 2. **Camera Mount Block (White):** 3D-printed to mirror the exact profile of the front chassis, providing a secure, rigid mounting point at the front of the vehicle to optimize the camera's field of view.
- **Steering Challenge:** Testing revealed that holding the DC steering motor at full lock caused severe mechanical strain, risking stripping the factory plastic gears.
- **Software Iteration:** Implemented software-level constraints to safeguard the drivetrain, limiting the steering power PWM and restricting the active pulse duration.
- **Observation:** Leveraged the chassis' built-in mechanical centring spring. By simply cutting power via `stopSteering()`, the physical spring snaps the wheels straight instantly. This configuration saves battery life and eliminates the need for active realignment motor pulses.

3. POWER AND SENSOR ARCHITECTURE

The Power Budget

The following table outlines the electrical requirements for our final system using the **11.1V** battery upgrade.

Approximation Note: All electrical currents and operating voltages listed are estimated parameters derived from manufacturer device datasheets under expected load conditions, rather than real-time telemetry measurements.

Table: Estimated Electrical Power Budget (Approximated)

Component	Operational voltage (V)	Avg. Current Draw (I_{avg})	Peak Current Draw (I_{peak})	Power Source/Rail
Raspberry Pi 3 (SBC)	5.0V(Regulated)	600 mA	2,500 mA	Dedicated 2-2slot Battery Pack
Pi Camera	5.0V (via Pi)	100 mA	200 mA	Shared via Raspberry Pi
Arduino Uno (MCU)	5.0V (via Driver)	50 mA	100 mA	Regulated 5V Output from L298N
Drive Motor	~7.9V – 9.3V (Effective)	250 mA	1,200 mA	Main 3-slot (11.1V) Battery Pack
Steering Motor	~7.9V – 9.3V (Effective)	150 mA	800 mA	Main 3-slot (11.1V) Battery Pack
TOTALS	N/A (Dual-Rail System)	1,150 mA	4,900 mA	Values approximated from data sheets

Battery Capacity & Runtime Justification

For our final build, we utilized a series configuration of three 1800mAh 18650 Li-ion cells to power the vehicle's drivetrain. To ensure the robot can complete the WRO mission requirements (typically 3 laps in under 3 minutes) without performance degradation, we calculated our safe runtime based on our updated power budget:

1. Theoretical Runtime:

$$Runtime = \frac{Battery\ Capacity}{Total\ Average\ Current} = \frac{1800\ mAh}{1150\ mA} \approx 1.57\ hours$$

2. Operational Reality:

In a high-performance WRO environment, voltage "sag" occurs as the battery pack drains. Even though the mathematical ideal suggests over an hour and a half of operational life, we have identified that consistent speed and "aggressive torque" are optimally maintained for the first 25 to 35 minutes of continuous use under load.

Sensor Selection, Placement & Calibration

1. Sensor Selection

We selected the Raspberry Pi Camera Module 3 NoIR Wide as our primary navigation sensor to meet the WRO requirement for advanced computer vision.

- **Trade-off Analysis:** Unlike ultrasonic sensors, which only provide distance data, a camera allows for semantic segmentation of the track. This is critical for distinguishing between red traffic signs (keep right) and green traffic signs (keep left) as defined in the Obstacle Challenge rules.
- **System Integration:** By using the Pi Camera Module 3, we could leverage the Raspberry Pi's processing power to run OpenCV-based algorithms, allowing the SBC to handle high-level visual logic while the Arduino MCU executes real-time motor commands.

2. Sensor Placement Justification

- **Orientation:** The camera is mounted completely flat and vertically straight on the custom 3D-printed front block.
- **Reasoning:** This position utilizes the module's native wide-angle Field of View (FOV) to see the track surface and upcoming markers simultaneously, allowing timely steering without overshooting turns.
- **Glare Mitigation:** Placing the camera low on the front block means the overhanging red deck shields the lens from direct overhead venue light glare, protecting colour threshold accuracy.

3. Calibration Process and Logic

Because WRO competition environments often have varying lighting and colour shifts, a robust calibration process is mandatory.

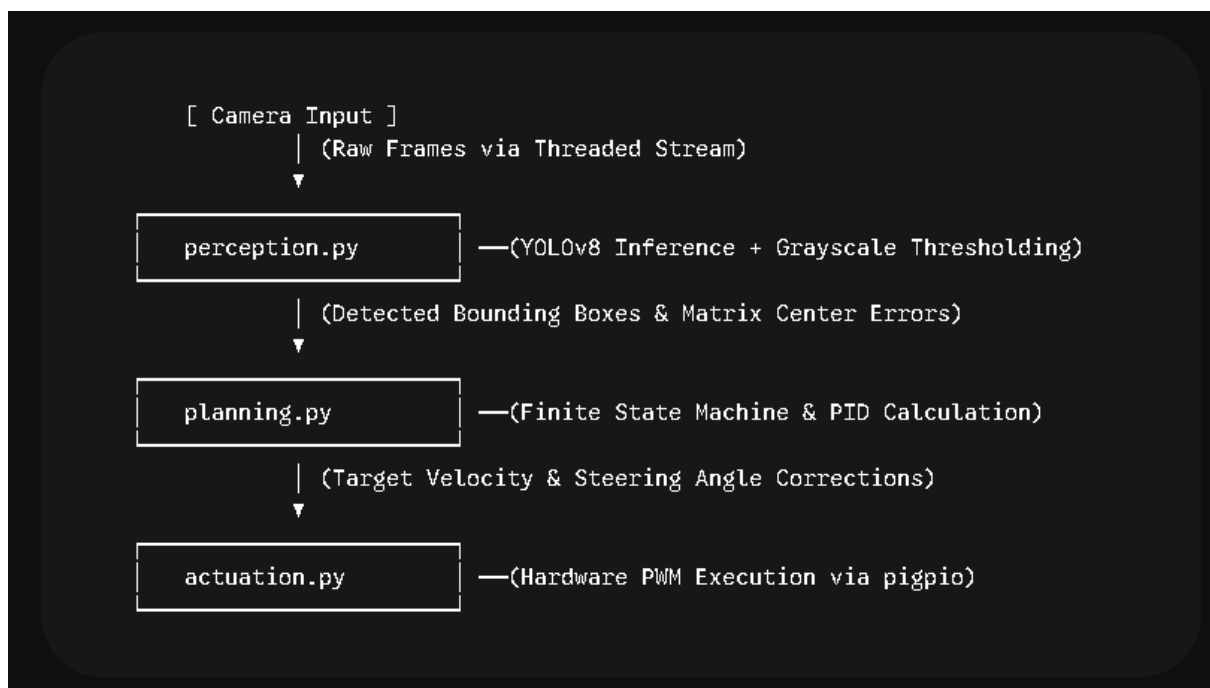
- **Dynamic HSV Thresholding:** Instead of hard-coding colour values, our calibration code uses HSV (Hue, Saturation, Value) thresholding. HSV is more resilient to light intensity changes than standard RGB.

4. SOFTWARE ARCHITECTURE AND OBSTACLE STRATEGY

3.1 Software Architecture & Modularity

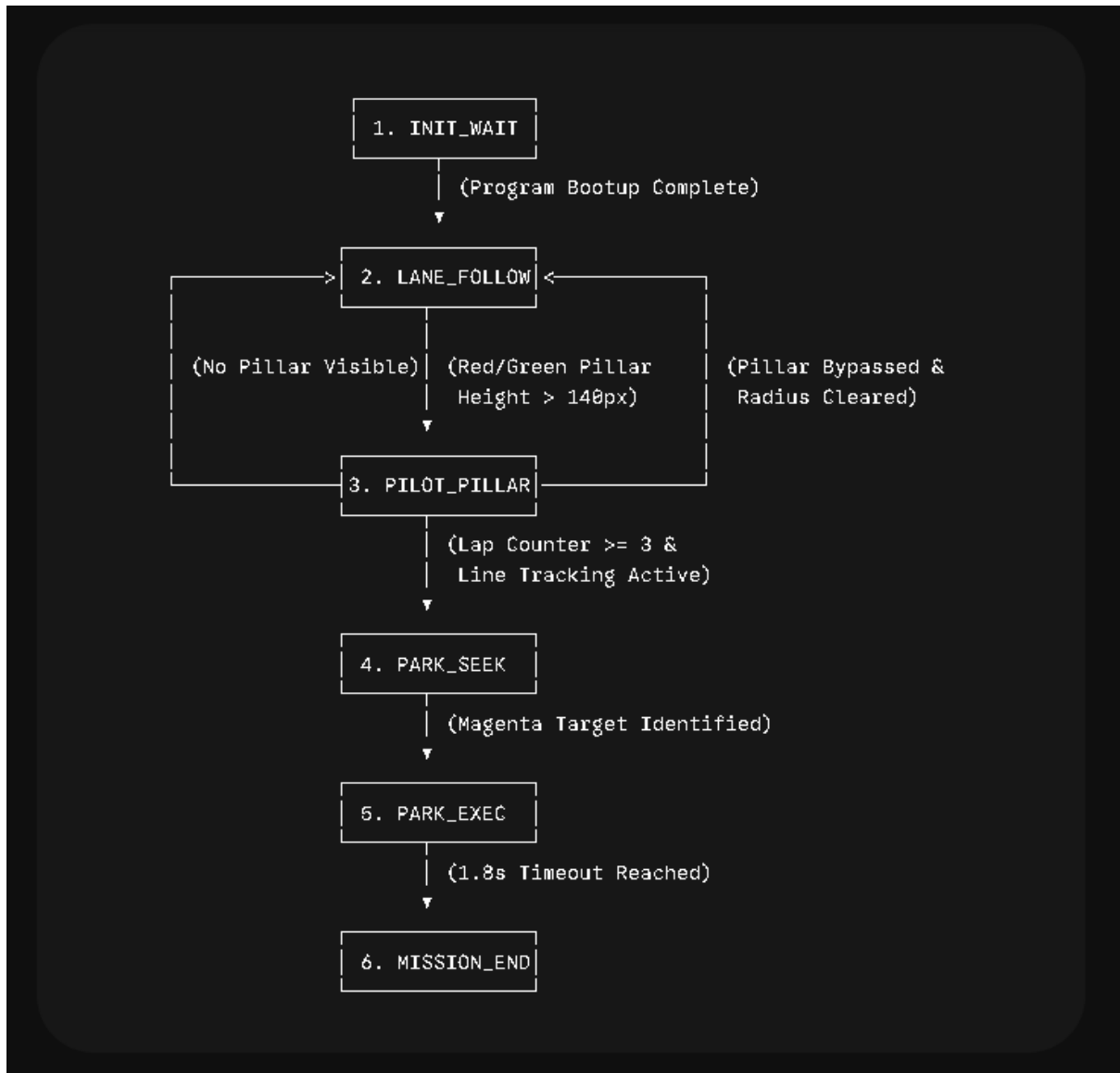
To maximize efficiency on the Raspberry Pi 4's multi-core ARM architecture, our control software is designed using a strictly modular micro-service paradigm. Instead of a single monolithic script, processing pipelines are split across independent Python modules interacting via synchronized thread-safe queues.

- `main.py`: The core orchestrator that initializes threads, handles global states, and acts as the system supervisor.
- `perception.py` (Vision Engine): Manages the hardware camera pipeline via OpenCV, captures frames, runs the custom-trained YOLOv8 inference engine, and outputs target coordinates.
- `planning.py` (Decision Engine): Governs high-level tracking logic, manages Finite State Machine transitions, and executes the mathematical PID calculations.
- `actuation.py` (Hardware Control): Directly interacts with the hardware-timed sockets (pigpiod) to map planned steering updates and velocities to physical PWM signals for the steering servo and drive motor.



3.2 Finite State Machine (FSM) Flow Logic

The vehicle's autonomous behavior transitions through a deterministic Finite State Machine (FSM). This guarantees predictable behavior when handling complex scenarios, such as moving from normal track navigation to overtaking maneuvers or searching for the final parking box.



Rationale for State Transitions

- **INIT_WAIT to LANE_FOLLOW:** Moves automatically out of standby upon complete instantiation of the hardware pins, minimizing delay.

- **LANE_FOLLOW to PILOT_PILLAR:** Triggered when the height of an incoming red or green bounding box exceeds 140 px. This transition shifts the core trajectory center target to bypass the obstacle safely.
- **PARK_SEEK to PARK_EXEC:** Once the lap tracker registers 3 full laps, the system filters for the distinctive magenta color signature of the parking area, initiating a precision slowing down and stopping sequence.

3.3 Lane Following & Obstacle Obedience Strategy

1. Computer Vision Pipeline (YOLOv8 + OpenCV)

Rather than classic, noise-sensitive color thresholding, our architecture leverages an ultra-lightweight, custom-trained **YOLOv8-nano** model optimized via **ONNX Runtime** to achieve low-latency inference directly on the Pi 4 CPU.

- **Classes Tracked:** red_pillar (Class 0), green_pillar (Class 1), magenta_wall (Class 2), black_wall (Class 3).
- **Fallback Centerline Estimation:** If explicit bounding boxes are distant, the perception engine uses a secondary OpenCV grayscale threshold pipeline on the lower half of the frame matrix (the immediate Region of Interest). It isolates the geometric balance point (moments) of the track space to provide an instant tracking deviation value ($e(t)$).

2. Obstacle Strategy & Side Obedience

To comply with the rule requiring red pillars to be bypassed on the right and green pillars on the left, the planner injects an offset directly into the tracking equations:

$$\text{Target Offset } (e_{sp}) = \begin{cases} \text{baseline_lane_error} + 65.0 & \text{if Green Pillar (Pass Left)} \\ \text{baseline_lane_error} - 65.0 & \text{if Red Pillar (Pass Right)} \end{cases}$$

This temporary modification shifts the setpoint used by the steering algorithm, causing the vehicle to move safely clear of the pillar without drifting out of bounds.

3. Closed-Loop Steering Control (PID)

Chassis heading corrections are governed by a continuous Proportional-Integral-Derivative (PID) loop:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

- **Proportional ($K_p = 0.42$):** Corrects aggressive deviation; scales structural steering angle smoothly based on instantaneous lane errors.
- **Integral ($K_i = 0.00$):** Purposefully zeroed to prevent integral windup, which causes dangerous over-corrections and oscillation loops when exiting sharp 90-degree corners.
- **Derivative ($K_d = 0.11$):** Dampens rapid steering changes, stabilizing the vehicle against tracking latency spikes.

5. SYSTEMS THINKING AND ENGINEERING DECISIONS

Trade-Off Analysis

We utilized a trade-off analysis to justify our move from the stock power system to a high-performance setup. The primary constraint was the **L298N driver's inherent 1.5V to 2.0V voltage drop**, which caused our initial prototype to stall

Component Choice	Option A: Stock 7.4V (2S)	Option B: Upgraded 11.1V	Decision & Justification
Torque Delivery	Poor; ~5.4V at motor was insufficient to break static friction.	Excellent; ~7.9V–9.3V effective voltage provided "aggressive torque".	Option B Chosen: Necessary for consistent acceleration on competition mat.
Thermal Load	Low; minimal heat generated.	High; the L298N internal chip dissipates significant heat, causing its integrated metal heatsink to become hot under heavy motor loads.	Accepted trade-off; relied on the driver's built-in metal heatsink for passive thermal dissipation, paired with software duty-cycle limits to avoid overheating.
System Stability	High risk of MCU brownouts due to voltage "sag".	Stable; high voltage headroom prevents resets during spikes.	Option B Chosen: Reliability is critical for autonomous mission success.
Physical Fit	Fits factory battery slot perfectly.	Exceeds factory slot volume; too large for the internal compartment.	Mounted externally on the main deck directly over the factory slot, keeping the original chassis completely intact.

Risk Mitigation

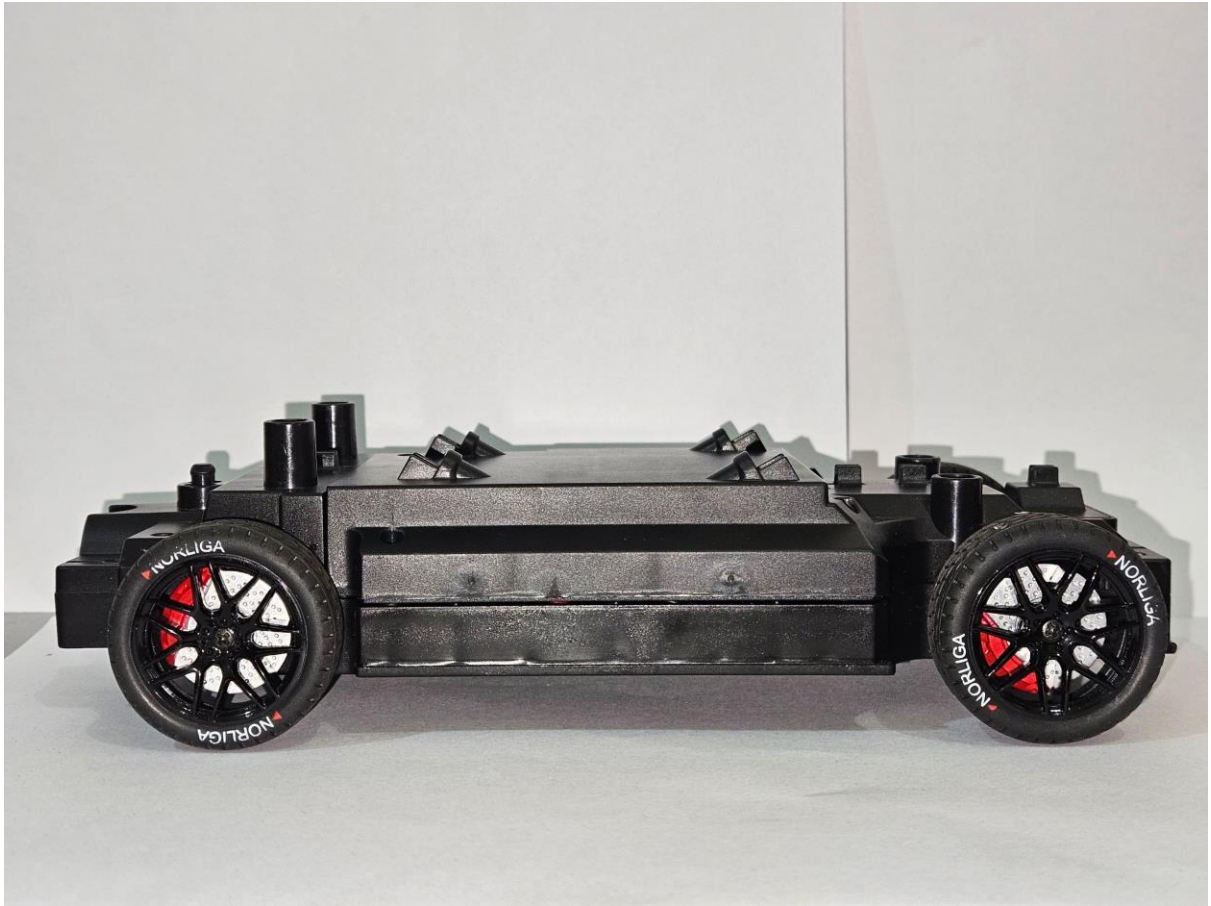
We identified critical failure modes that could lead to disqualification or mechanical damage and implemented engineered mitigation strategies for each.

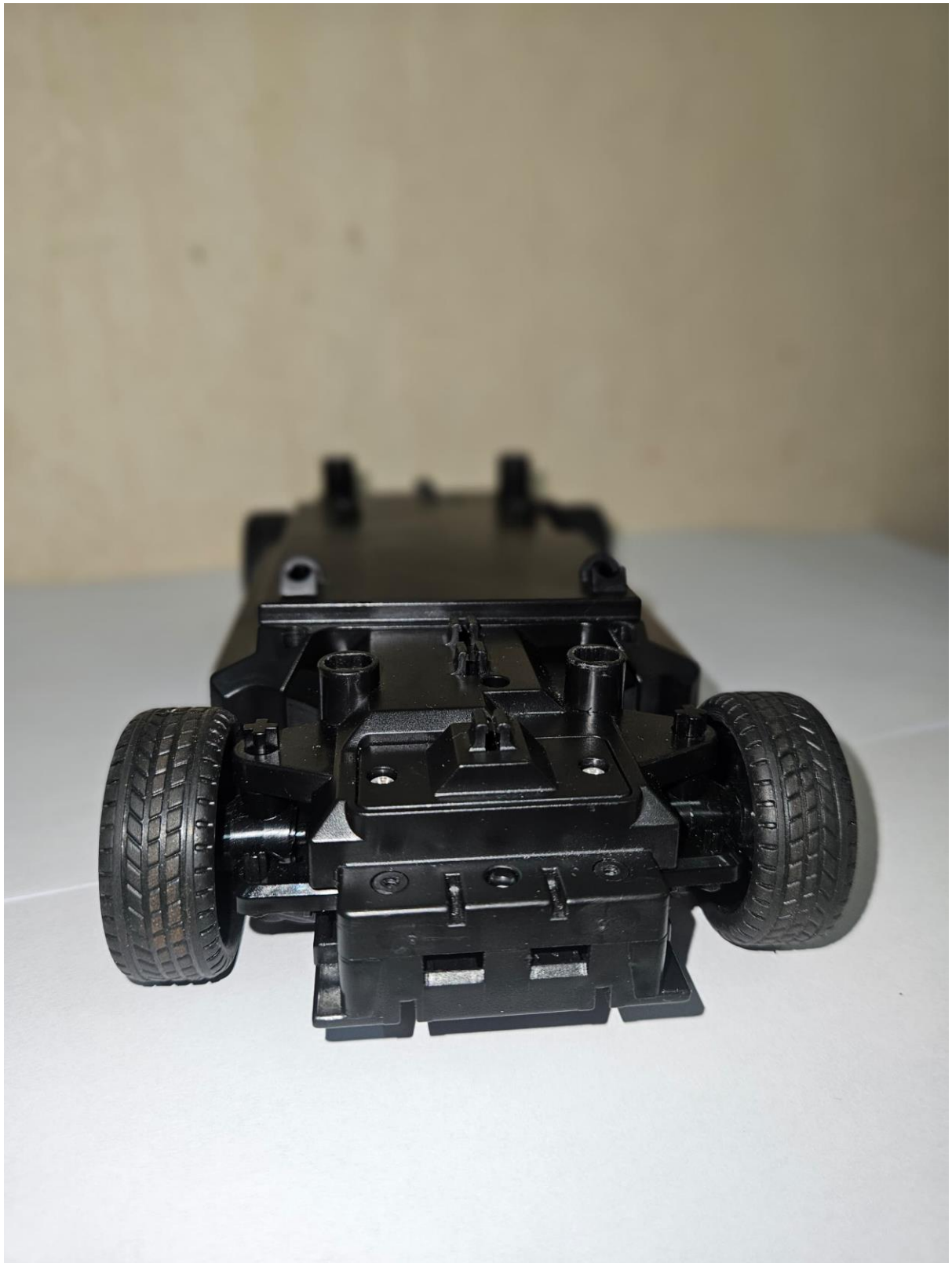
Potential Risk	Effect on Performance	Engineering Mitigation Strategy
----------------	-----------------------	---------------------------------

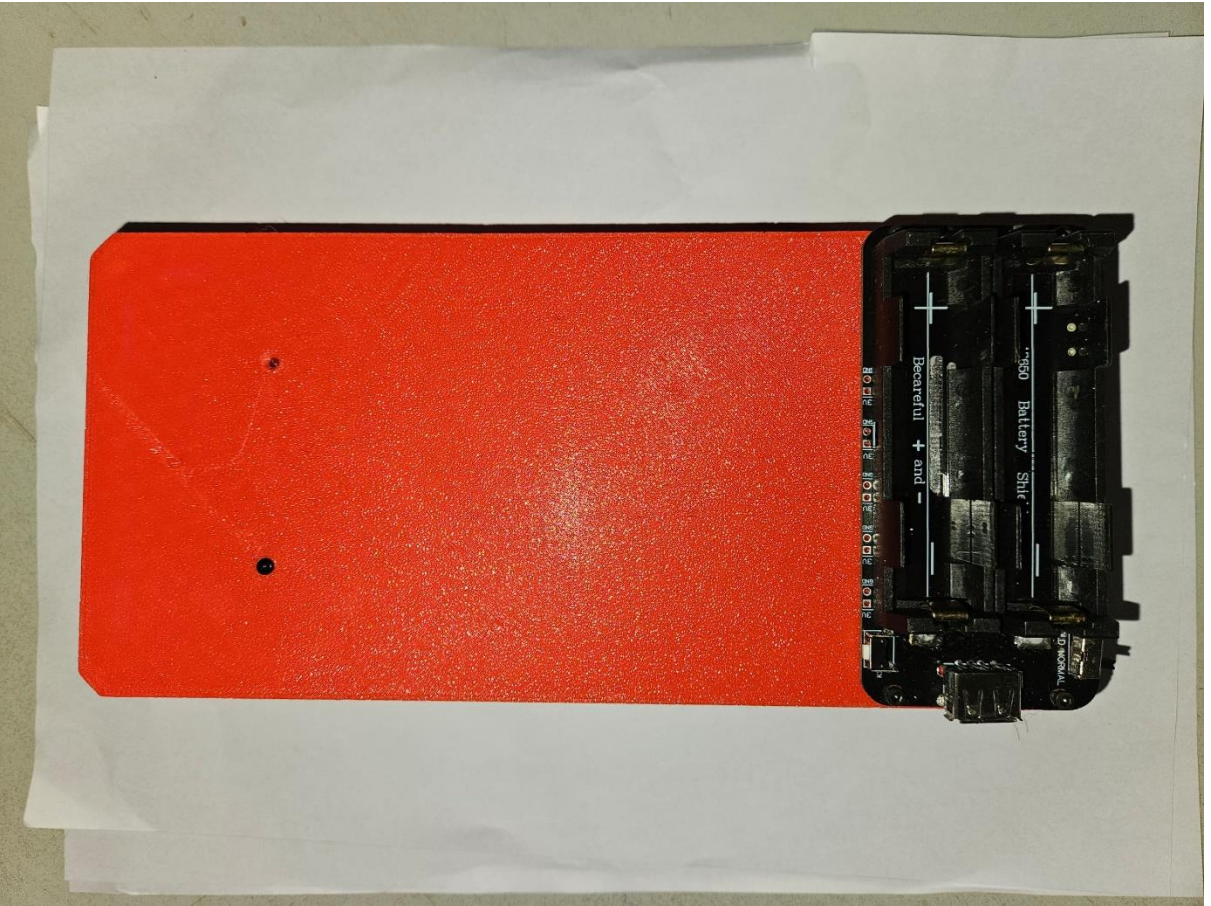
Ambient Light changes	Glare or shadows throw off colour detection of pillars.	Executed a baseline calibration script during practice time to map and save venue-specific colour values into the navigation software local memory.
Mechanical Steering Strain	DC steering motor fights physical limits, risking stripped factory gears.	Software Constraints: Hard-coded software limits to restrict active steering power and pulse duration.
Power Sharing Noise	Heavy motor current draw spikes cause voltage sags, crashing the processing boards.	Power Rail Isolation: Completely isolated the Raspberry Pi 3 on a separate 2-slot battery rail, while the Arduino utilizes the L298N's regulated 5V output terminal.
Mechanical Binding	High internal gearbox friction leads to motor stalls or electrical overheating.	Pre-Round Inspection: Implemented a mandatory pre-run checklist including manual wheel-spin tests and drivetrain alignment checks.

SHOTS OF OUR AUTONOMOUS VEHICLE.

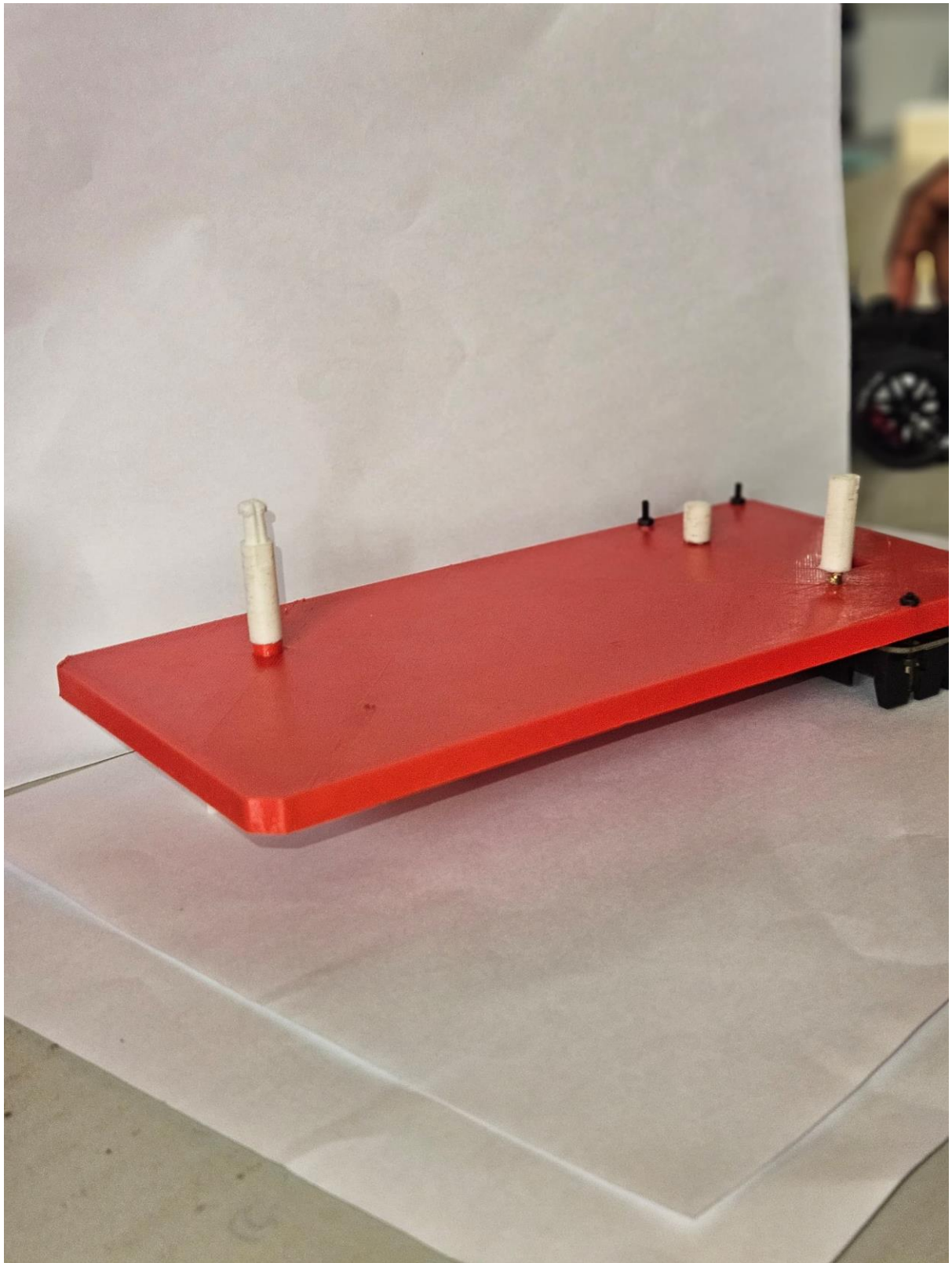


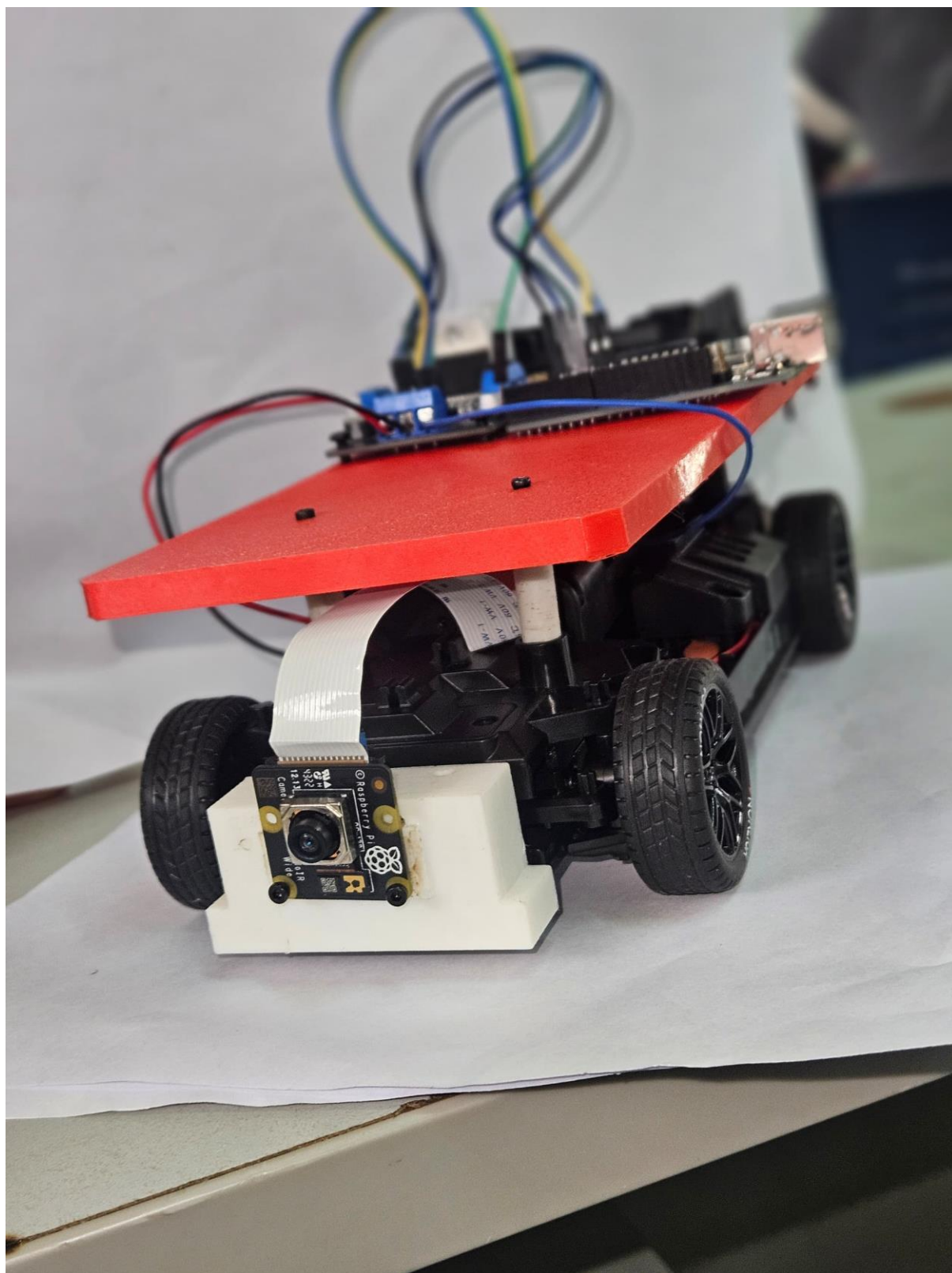


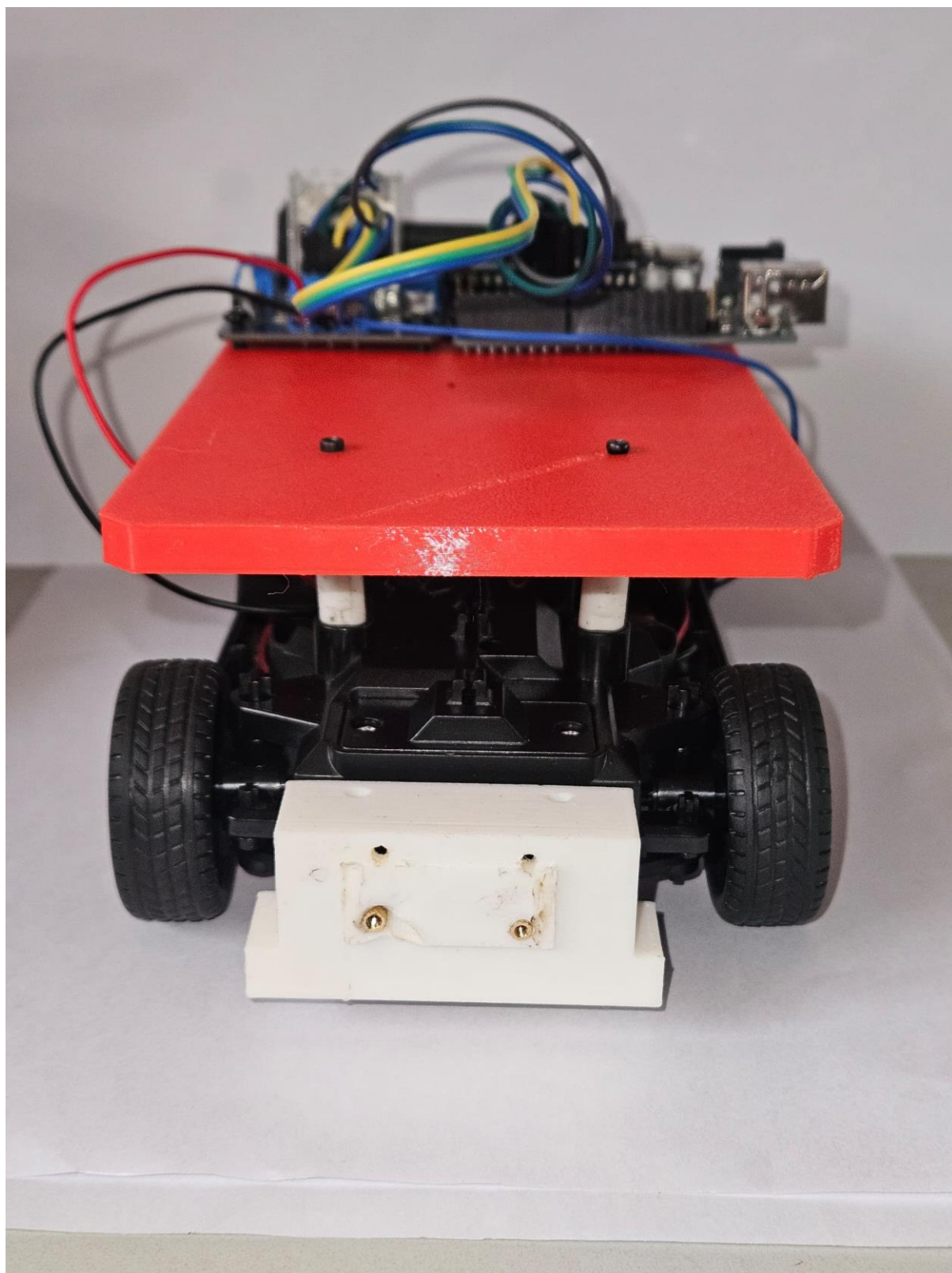




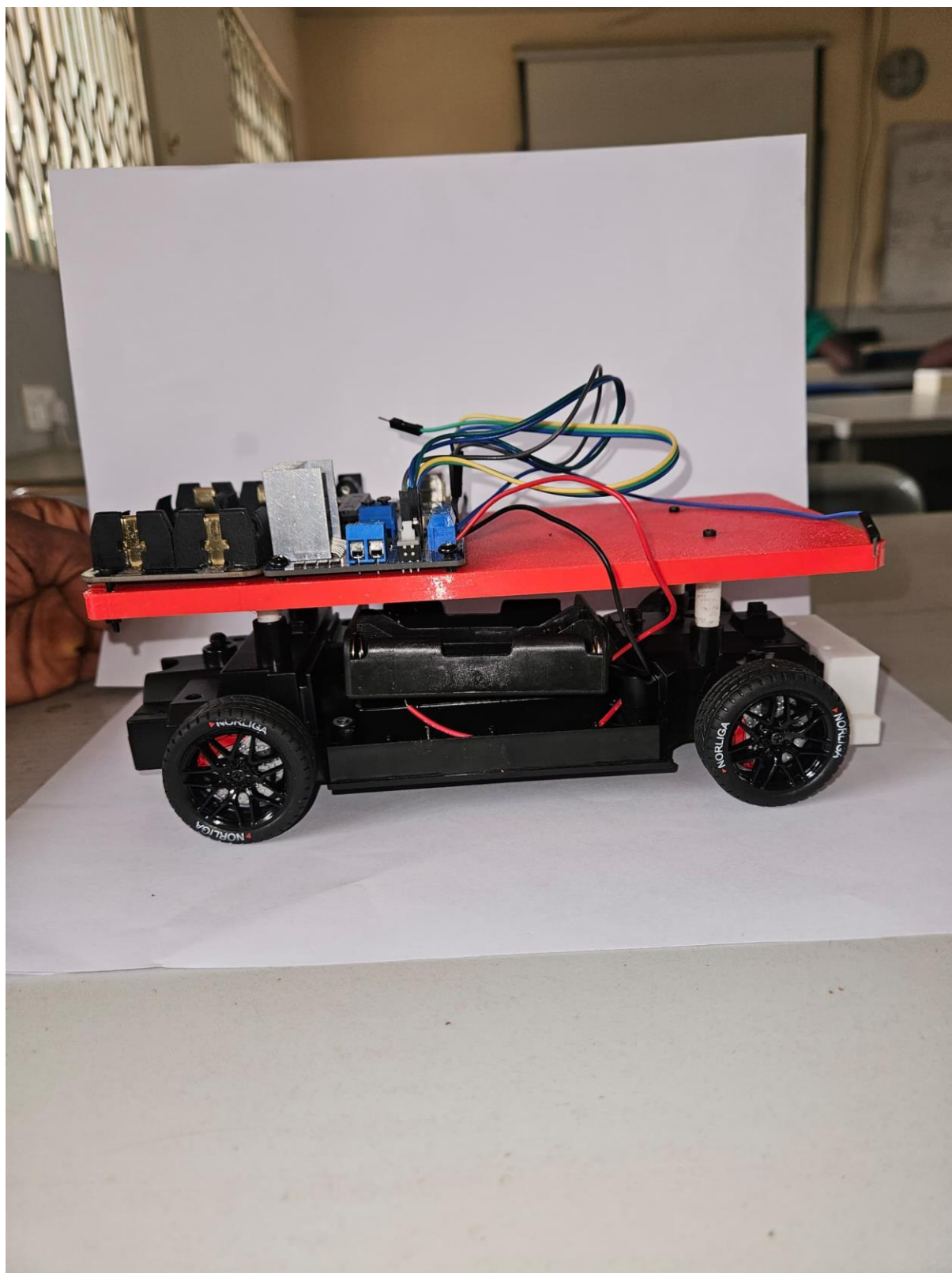


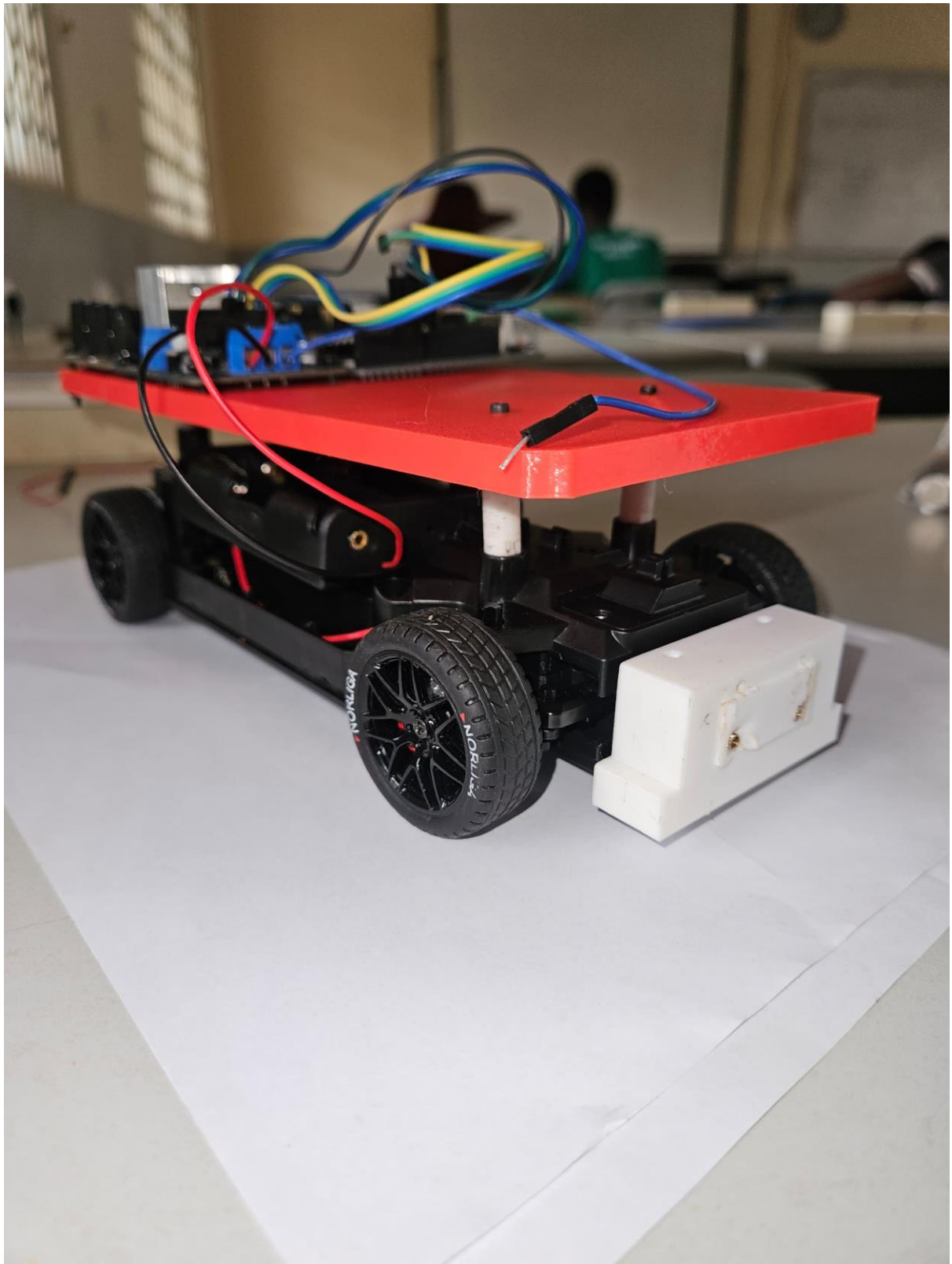




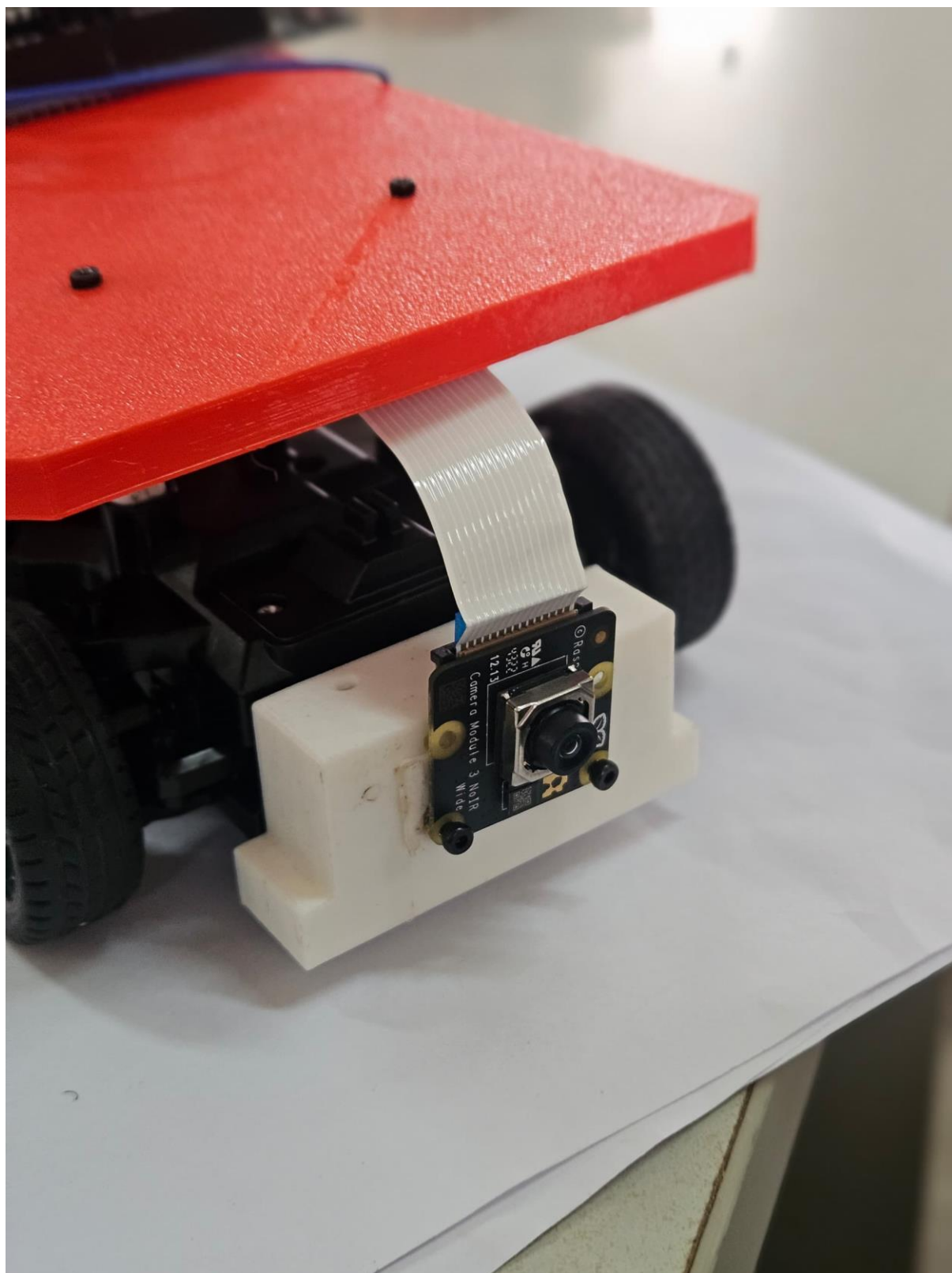


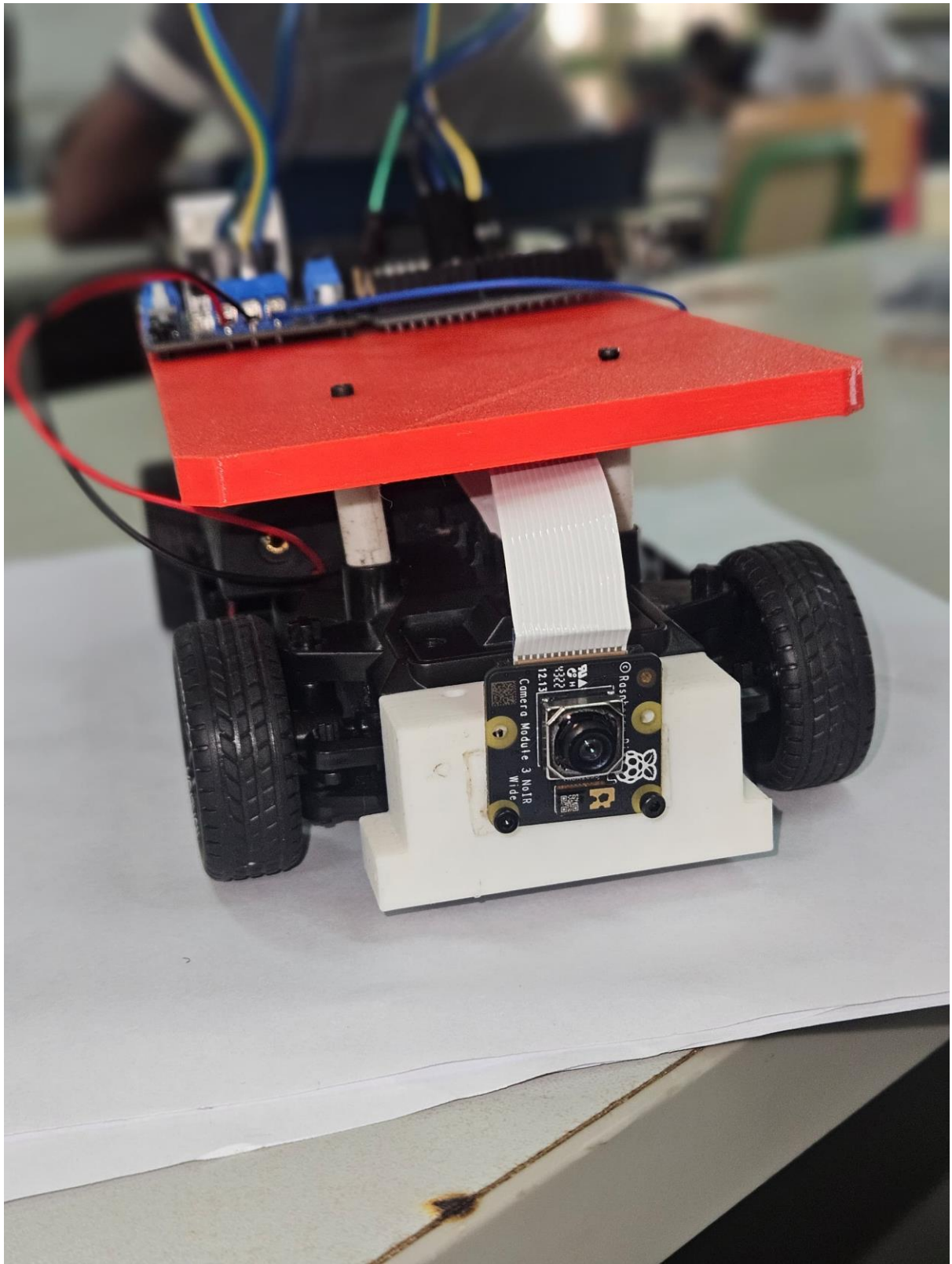












REFERENCE

1. Gillespie, T. D. (1992). Fundamentals of Vehicle Dynamics. SAE International.
2. Siegwart, R., Nourbakhsh, I. R., & Scaramuzza, D. (2011). Introduction to Autonomous Mobile Robots (2nd ed.). MIT Press.
3. World Robot Olympiad. (2026). Future Engineers 2026 - Documentation Rubric.
4. Bird, J., 2017. Electrical Circuit Theory and Technology. 6th ed. London: Routledge.
5. STMicroelectronics, 2000. L298: Dual Full-Bridge Driver. [Datasheet]. Geneva: STMicroelectronics. Available at: <https://www.st.com/resource/en/datasheet/l298.pdf>.